IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Manuscript prepared for resubmission, October 30, 2026.

Latency, Numerical Fidelity, and Resource Tradeoffs in Mixed-Precision Robot Policy Inference

Xiongyule Zhou

School of Mechanical, Electronic and Control Engineering, Beijing Jiaotong University, Beijing 100044, China
Corresponding author: Xiongyule Zhou (e-mail: jensenzhou47@gmail.com).

ABSTRACT Robot policy acceleration requires a configuration that satisfies numerical and kinematic constraints within a defined resource budget. We designed and conducted paired inference experiments using robot observations, component-level precision ablations, and fixed-input backend comparisons. The evaluation separates 30-, 50-, and 60-position action chunks, matches observations and diffusion noise where complete identifiers are available, and distinguishes forward-call timing from preparation and output transfer. On a 32-input sequence, combined dynamic and static mixed-precision configurations reduced median latency by 25.58% and 38.38%, respectively, but minimum input cosine fell to 0.928032 and 0.965292. A separate four-camera, 26-coordinate experiment jointly measures native-action and nominal flange discrepancies. Against its local PyTorch reference, TensorRT denoiser acceleration reduced prepared-call median latency from 255.51 to 238.64 ms, with a maximum flange difference of 3.756 mm. The graph-enabled split configuration reached 175.27 ms, a 31.40% reduction, but increased the maximum difference to 5.314 mm and the PyTorch process-peak allocation counter from 8.715 to 8.845 GiB. With a 9-GiB counter budget, a finite-input screen of five representative configurations selects the denoiser under 5-mm/0.5-degree discrepancy limits and the graph-enabled split under 6-mm/0.75-degree limits. These illustrative limits constrain reference agreement, not physical task performance. The results support selective acceleration and configuration-specific validation; they do not establish closed-loop task equivalence or comparative energy savings.

INDEX TERMS Edge computing, mixed precision, post-training quantization, robot learning, vision-language-action models.

I. INTRODUCTION

Vision-language-action (VLA) policies combine visual observations, language instructions, and proprioception to generate action chunks. OpenVLA and π0.5 extend this approach across manipulation settings and embodiments [1], [2]. Onboard computation makes inference latency and resource cost practical deployment constraints. Precision selection must therefore account for how acceleration changes the numerical actions presented to a controller.

Post-training quantization (PTQ) provides one route to reducing arithmetic and data movement. However, the nominal precision of a matrix multiplication does not specify the behavior of a complete policy. A deployment can combine low-precision projections, higher-precision normalization, calibration scales, runtime quantization, fused kernels, and cached weights. Implementation changes can also alter output values before any reduction in nominal precision occurs. Consequently, a reported speed ratio needs an explicit reference implementation and measurement boundary. We consider bfloat16 (BF16), eight-bit floating point (FP8), and NVIDIA four-bit floating point (NVFP4) configurations.

Robot-specific quantization methods address decision-critical states, action sensitivity, and denoising dynamics [3]-[6]. An unresolved deployment question is how to select among component precisions and execution backends when output similarity, nominal joint geometry, latency, and resource counters favor different configurations.

A useful experiment must control the quantities that can otherwise change the answer. Observations and diffusion noise determine the comparison population. The reference implementation defines numerical discrepancy. Forward execution, preprocessing, output transfer, weight storage, and runtime allocation require separate measurement boundaries.

Repeated timings on one input characterize a different quantity from fidelity across many inputs.

We designed and performed inference experiments with π0.5 implementations and collected robot observations, action outputs, execution timings, and resource measurements. Component ablations compare BF16, FP8, and NVFP4 paths. A four-camera checkpoint supports joint evaluation of latency, native-action fidelity, and nominal forward kinematics. Additional experiments examine a separate action schema, key-value (KV) cache precision, and complete-cycle resource costs [7].

The study asks which component changes preserve input-level agreement, how nominal flange discrepancy alters a latency-based choice, and which resource measures constrain that choice. We answer these questions within each experimental group, using its own reference, input population, and timer. Cross-group comparisons identify measurement requirements rather than a universal speed ranking.

The contribution is an experimental evaluation and a constraint-based configuration screen. It links paired observations and noise to action discrepancy, relates action changes to nominal flange geometry, and jointly reports latency and allocation counters for a common checkpoint. The analysis preserves adverse results and defines the conditions under which a faster configuration remains eligible.

II. RELATED WORK

A. QUANTIZATION AND IMPLEMENTATION COST

General PTQ methods differ in what they preserve and where their costs occur. GPTQ uses approximate second-order information for weight quantization [8]. Activation-aware Weight Quantization (AWQ) uses activation statistics to protect salient weight channels through scaling [9]. SmoothQuant redistributes activation outliers through an equivalent transformation to facilitate low-precision computation [10]. These methods provide relevant tools, but language-model evaluations do not establish robot task performance.

Low-bit arithmetic also requires a compatible implementation. Triton documents block-scaled arithmetic and its packing and scaling requirements [11]. NVIDIA describes supported quantized representations and scaling rules [12]. Our A, B, J60, and D experiments use Triton-based paths; E/F compare PyTorch and TensorRT (TRT) configurations. The separate LingBot-VA experiment uses an FP8 core and paged attention. These distinct precision scopes require configuration-specific measurements.

B. ACTION-SENSITIVE ROBOT POLICY QUANTIZATION

SQIL links quantization to decision-critical states through saliency-aware training and evaluates policy outcomes [3]. QVLA measures action sensitivity to guide channel-level precision allocation [4]. HoloQ-VLA combines complementary rotations with denoising-step scaling for uniform low-bit quantization [5]. CHASE-VLA adapts action-expert activation scales using previously generated chunks and denoising-step information [6]. Its action-expert timing boundary differs from a complete policy call, illustrating why published speed ratios require compatible denominators.

Our experiments complement action-sensitive quantization methods by evaluating practical configuration choices. We do not implement these methods as competing algorithms or compare their task-success rates. Their reported speed ratios use different models, hardware, and timing scopes. Our comparison instead fixes an input population within each group and examines numerical and nominal geometric agreement together.

LingBot-VA combines video prediction and action generation with persistent context [7]. FlashInfer supports paged attention across cache layouts and shapes [13]. These systems motivate the cache and complete-cycle experiments. A cache-kernel comparison and a whole-policy stream address different workloads, so kernel acceleration is not extrapolated to a policy speedup.

C. MEASUREMENT AND UNCERTAINTY

MLPerf Inference emphasizes explicit workloads and measurement rules [14]. Kalibera and Jones distinguish repeated executions from within-execution iterations [15]. Reinforcement-learning evaluation also requires correctly characterized repetition [16]. We report descriptive effects over the tested inputs and repeated timing calls. Action coordinates and predicted positions are nested measurements, not independent experimental trials.

III. EXPERIMENTAL DESIGN AND METHODS

A. EXPERIMENTAL GROUPS AND DATA COLLECTION

The author designed and implemented the inference experiments and collected the observations and measurements used here. Real-observation groups cover cup transfer, bottle transfer, and packing contexts. Synthetic fixtures are restricted to diagnostic or timing experiments. Table I specifies each group, its action shape, and its input-pairing condition.

Each configuration is compared with a named reference within its experimental group. We preserve input order, action-coordinate order, preprocessing definitions, and diffusion noise when the experiment provides them. SHA-256 checksums bind the input packages and outputs. An execution repeat with identical outputs adds timing information without adding another observation or task.

We distinguish observation-based fidelity evaluation from repeated-input benchmarking. Camera and state samples describe the policy inputs; candidate action tensors describe predictions on those inputs. No physical task-success or tracking measurement is included. The data support inference fidelity and nominal kinematic analysis without a claim of closed-loop behavioral equivalence.

Tensor-resolved groups permit direct recomputation of discrepancies. The J60 benchmark supplies configuration summaries and fixture checksums, with raw action and timing vectors unavailable. We identify this limitation at the corresponding results. Configuration and software records

specify the intended execution paths; missing run-level details remain explicit rather than being assumed constant.

TABLE I. Experimental groups, input coverage, and pairing conditions.

Group	Input and action dimensions	Experimental scope and pairing
A/B	32 + 32 observations; H30, D32	Complete input-package hashes include images, states and diffusion noise.
Diagnostics	11 constructed inputs; a 44-input repetition	Synthetic state sweeps and repeated inputs; excluded from real-observation count.
J60	96 calibration / 16 selection / 45 validation inputs	45-input fidelity summaries; 50 timed calls on one prepared input. Raw vectors unavailable.
D	115 inputs; H60, D32	Ordered identifier pairing; complete images/noise digests unavailable.
E	32 observations; H50, D26	Four cameras; explicit noise and coordinate names. Optimization summaries identify the complete reference package.
F	32 observations; H50, D16	Order/seed pairing; incomplete noise linkage and unresolved gripper-column schema conflict.
Systems	Three synthetic cache cases; one 160-cycle stream	Separate kernel timing and steady-cycle accounting; no matched whole-policy precision baseline.

B. OBSERVATIONS AND INPUT-NOISE PAIRING

Groups A and B each contain 32 distinct image, state, and noise tensors. Their state-source timestamps span 6.201 and 9.710 s, respectively. Complete input-package hashes begin with 7f3fa817 and 683de797; full digests accompany the analysis. These short sequences retain temporal dependence and do not represent 64 independent tasks.

An 11-input diagnostic fixture contains three distinct image tensors and nine distinct state vectors. Its first three states are zero, and state-sweep entries have zero source timestamps. A 44-input diagnostic repeats the same 11 image-state-noise combinations four times. These constructed inputs are excluded from the count of real observations.

The J60 experiment uses 96 calibration inputs from 32 recordings, 16 selection inputs from eight recordings, and 45 validation inputs from 45 recordings. All 85 recording fixtures match the dataset-manifest checksums. Recording names and exact image-state pairs do not overlap across these splits. The cup-transfer contexts cover both movement directions in calibration and selection, and one direction in validation. These development splits do not establish an independently blinded test.

The component and profile screens include additional A/B configurations on the same input packages. Sixteen B outputs and seven A outputs preserve their corresponding complete fixture hashes. Three A output pairs duplicate other configurations, so they are treated as aliases. These comparisons extend configuration coverage without increasing the input population.

Group D evaluates 115 ordered sample identifiers with action shape 115 × 60 × 32. Its strata contain 45 validation recording/frame pairs, 60 inputs from 54 additional recording names, and ten inputs from five diagnostic scenes. The first stratum overlaps J60 validation identifiers. Identifier equality does not authenticate equal images, preprocessing, or noise; D is therefore a separate, conditionally paired comparison.

C. POLICY AND CONFIGURATIONS

The policy configuration for A, B, J60, and D has 27 vision layers, 18 language-encoder layers, 18 action-decoder layers, 32 action coordinates, and ten denoising steps. Each observation contains three RGB views of size 3 × 224 × 224. Experiments with 30- and 60-position chunks are separate workloads. Changing chunk length does not create another timing repetition of the same workload.

The A comparison uses eager and static BF16 references. Ablations change the vision path, encoder feed-forward network (FFN), attention path, or decoder, followed by dynamic-FFN and combined configurations. Component labels identify the intended changed path. The complete executable and clock state are not matched sufficiently to attribute every latency difference solely to precision.

For J60, dynamic FP8 covers vision, encoder FFNs, encoder attention, and the decoder. A second dynamic profile adds NVFP4 decoder FFNs while retaining encoder FFNs in FP8. The static profile combines calibrated FP8 paths with AWQ NVFP4 encoder FFNs in layers 0-16. Layer 17 FFN is not executed in this graph. Calibration uses scale margin 1.1, AWQ $\alpha = 0.5$, and percentile 100.

D. NUMERICAL FIDELITY

Let R and C denote reference and candidate tensors with shape N × H × D. A/B require equal complete input hashes, dimensions, and ordering, including diffusion noise. E compares explicit noise and coordinate names; its optimized configurations also carry companion summaries identifying the complete reference package. D/F have weaker identifier or order/seed pairing and remain separate. We compute all discrepancy metrics in float64. Mean absolute error (MAE) is

$$MAE(R,C)=\frac{1}{NHD}\sum_{i=1}^{N}\sum_{h=1}^{H}\sum_{d=1}^{D}|C_{ihd}-R_{ihd}|. \quad (1)$$

We also calculate root mean squared error, relative L2 discrepancy, coordinate-error quantiles, and maximum absolute discrepancy. Aggregate cosine is

$$c_{all} = \frac{\langle \mathrm{vec}(R), \mathrm{vec}(C) \rangle}{\| \mathrm{vec}(R) \|_2 \, \| \mathrm{vec}(C) \|_2}. \tag{2}$$

Minimum input cosine is the smallest cosine across complete predicted action chunks. We preserve the distribution over inputs and the error at each predicted position. This prevents pooled agreement from concealing a localized input or coordinate discrepancy.

Normalized and native-coordinate outputs are evaluated separately. The 32-coordinate schema includes trunk, arms, head, and hands. Not every coordinate has a fully established physical unit and command contract, so pooled native-coordinate error is not interpreted as physical accuracy. Kinematic evaluation is restricted to named revolute joints with an explicit mapping and nominal geometry.

The numerical screen requires aggregate cosine at least 0.998, normalized MAE at most 0.03, and minimum input cosine at least 0.99. Alternative input floors of 0.98, 0.995, and 0.999 test sensitivity. These are reference-agreement criteria. They have no validated mapping to task success or robot safety.

E. NATIVE ACTIONS AND NOMINAL KINEMATICS

E evaluates 32 observations with four 3 × 480 × 848 uint8 images, a state vector, a task instruction, and explicit diffusion noise. Outputs contain 50 positions and 26 coordinates, comprising seven arm joints and six hand coordinates per side. The task concerns left-hand bottle transfer between two coasters. The cross-device reference is labeled NVIDIA H20, PyTorch 2.7.1+cu126, BF16. Local optimization uses NVIDIA Thor, PyTorch 2.9.1+cu130, CUDA 13.0, and 14 CPU threads. Candidates share the reference-package digest and diffusion noise. Eleven eligible output files contain four distinct tensors on the same observations. The policy configuration specifies BF16 and ten denoising steps; a complete operator-wise precision map for the TensorRT engines is unavailable.

F uses a separate three-camera checkpoint, 32 observations, and 50-position chunks with seven joints and one gripper coordinate per arm. Its task concerns packing clothes and a box into a bag. The implementation converts gripper widths to training fractions and generates noise from a seed. Candidate arrays lack complete input/noise digests. The fixture also conflicts with the assumed gripper-column range. F therefore supplies a conditional schema diagnostic and does not determine the recommended precision configuration.

We calculate nominal forward kinematics from the Unified Robot Description Format (URDF) joint tree, using float64 joint-origin and axis-angle transforms. Action columns are zero-based: E right-arm joints occupy columns 7-13, whereas F uses 8-14. For each flange, we obtain the position and orientation of the reference and candidate transforms. Their discrepancies are

$$d_p = 10^3 \, \| \, p_e(q_C) - p_e(q_R) \, \|_2 \, (mm),$$
$$d_R = \angle \left(R_e(q_R)^{\mathsf{T}} \, R_e(q_C) \right). \tag{3}$$

Orientation discrepancy is reported in degrees. Unspecified common upstream joints are set to zero; a shared upstream rigid transform leaves both discrepancy measures invariant. E uses the air-5 geometry and F uses air-3. A second geometry model gives identical E discrepancies for the selected comparisons. Scalar Rodrigues calculations independently reproduce batched transforms on 20 seeded poses per arm, with maximum matrix-element difference below 10^{-12}.

The primary kinematic screen counts inputs whose two flanges remain within 3 mm and 0.5° at every predicted position. Position-only sensitivity uses 1, 2, 5, and 10 mm. Configuration selection requires all 32 inputs to satisfy the position and orientation limits, then minimizes prepared-call median latency under the counter ceiling. We evaluate 3-mm/0.5°, 5-mm/0.5°, and 6-mm/0.75° scenarios with a 9-GiB PyTorch counter ceiling. These illustrative numerical budgets exclude physical tracking, contact, compliance, and geometric uncertainty.

F. LATENCY, REPETITION, AND RESOURCES

A contains one latency value for each of 32 inputs. We report median, mean, standard deviation, percentiles, and extrema; percentiles use linear interpolation. Synchronization and preprocessing coverage are not uniformly specified for these executions. Accordingly, A timings are reported within their per-input measurement boundary and are not treated as complete sensor-to-action latency.

J60 prepares its first input once, performs ten warmup forward calls, and times 50 forward calls using CUDA events. Fidelity is measured separately across 45 validation fixtures. The runtime records specify Jetson Thor in MAXN mode, but do not include synchronized clock, temperature, occupancy, or power traces. We do not pool this timer with A or E.

Storage is the sum of serialized weight-file bytes, reported in decimal gigabytes. Runtime allocation is reported in binary GiB from the named PyTorch counter. File storage, logical cache bytes, allocator peaks, total device memory, and rail energy are separate quantities. Weight and timing records are not assumed to describe an identical build when their revision identifiers differ.

E uses two prepared-call boundaries. The first timer covers preprocessing, forward execution, and postprocessing after reset, input cloning, and initial synchronization. It reuses noise from input zero during timing. Its 200-call comparisons and 150-call graph ladder follow ten warmups. The optimization experiment instead cycles observations with their own noise, uses synchronized wall-clock boundaries, and includes postprocessing and CPU output transfer. Each optimized configuration has ten warmups and 200 measured calls. Its PyTorch peak is a process-lifetime counter that can include model loading and omits some TensorRT allocations. Only summary percentiles are available, so timing confidence intervals cannot be reconstructed. F synthetic-input timing vectors are excluded from the real-input configuration selection.

G. SEPARATE CACHE AND COMPLETE-CYCLE RESOURCE CASE

The cache experiment uses BF16 queries and BF16 or FP8-E4M3 key-value tensors with batch/query pairs (1,24), (2,24), and (2,240). All cases use 9,504 context tokens, 24 heads, head dimension 128, and page size 16. Seeded synthetic tensors isolate the attention workload. FlashInfer 0.6.17 with the fa2 backend supplies five timing samples, each averaging 200 CUDA-event iterations after 50 warmups. Attention discrepancies are summary measurements; raw attention tensors are unavailable.

A separate FP8-core/BF16-cache stream contains 160 inference cycles. We verify action-array checksums and timestamp ordering, deduplicating rank-level reports. Cycles 60-159 define the 100-cycle steady region. Previous model outputs provide state backfill, and the benchmark excludes robot command publication. We measure complete-cycle, inference, and cache-update durations, with PyTorch allocated and reserved counters reported separately.

The stream includes 4,151 timestamped tegrastats observations. We integrate the first VDD_GPU power value reported at each timestamp over fully covered steady-cycle intervals using boundary interpolation and the trapezoidal rule [17]. Left-held integration tests interpolation sensitivity. These measurements include idle and other GPU-rail load; they lack idle subtraction, sensor calibration, and whole-board coverage. Without a matched BF16 stream, rail observations cannot establish energy savings.

H. STATISTICAL SCOPE AND REPRODUCIBILITY

The unit for primary discrepancy analysis is a paired recorded input, with temporal dependence within each sequence. Horizon positions and action coordinates remain nested within an input. We report finite-corpus descriptive effects, without treating coordinates as independent observations or assigning population-level significance to configuration differences.

We remove aligned contiguous blocks of one, four, or eight A inputs and recompute MAE. The resulting ranges measure sequence-composition sensitivity, not sampling confidence intervals. Technical timing repetitions and configuration aliases do not multiply the observation count. Supplementary derived tables and verification routines support the reported calculations; raw observations and complete action arrays remain outside the public release.

IV. RESULTS

A. REFERENCE CHOICE REVEALS IMPLEMENTATION DISCREPANCY

The eager and static BF16 outputs differed on sequence A despite the shared input hash. Their aggregate normalized cosine was 0.999198, and normalized MAE was 0.009347. Minimum input cosine was 0.987991. The maximum native-coordinate discrepancy was 0.049805. This establishes a numerical implementation discrepancy before low-precision configurations are evaluated; it does not identify a faulty implementation.

The recorded medians were 249.90 ms for eager inference and 109.87 ms for static BF16. Their ratio was 2.275. Because the implementations differ, this ratio should not be attributed to quantization. Subsequent primary fidelity results use static BF16 as the reference, while the supplied tables retain eager-reference metrics for comparison.

TABLE II. Thirty-position component and mixed-precision comparison on group A. Each configuration uses 32 hash-matched inputs. Speed ratio uses static BF16 as the timing reference; fidelity uses its outputs.

Configuration	Median (ms)	P95 (ms)	Speed ratio	Cosine	MAE	Min. input cosine	Inputs below 0.99
Eager	249.90	254.38	0.440	0.999198	0.009347	0.987991	1
BF16	109.87	110.32	1.000	1.000000	0.000000	1.000000	0
Vision only	105.63	108.98	1.040	0.999560	0.005076	0.987160	1
Encoder FFN only	101.63	102.46	1.081	0.996780	0.019490	0.978087	2
Attention only	110.51	113.14	0.994	0.999252	0.009210	0.992153	0
Dynamic FFN	102.39	103.08	1.073	0.999420	0.008535	0.996273	0
Decoder NVFP4	101.30	101.98	1.085	0.998229	0.015921	0.993852	0
Dynamic mix	81.76	81.94	1.344	0.995056	0.022945	0.928032	5
Static mix	67.70	68.00	1.623	0.996464	0.022513	0.965292	2

Speed ratio is the BF16 median divided by the configuration median. Eager and static BF16 differ in implementation while using BF16 arithmetic. Component labels identify the intended changed path. Full identifiers, P99 values, extrema and both reference choices accompany the supplementary data.

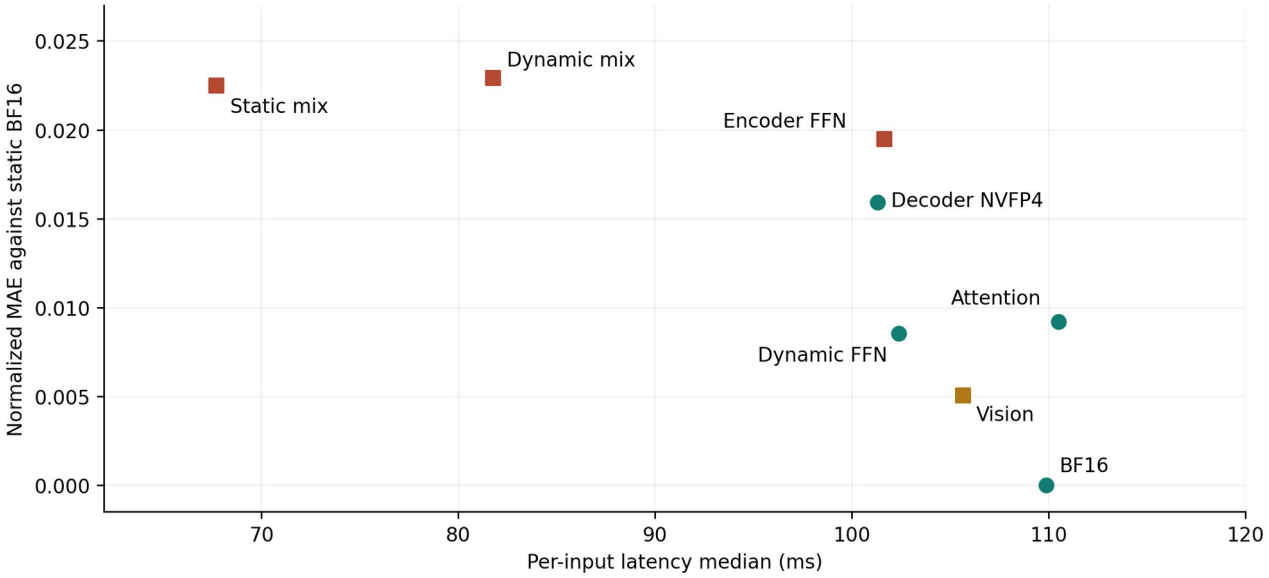


FIGURE 1. Latency and normalized output discrepancy for group A. Points correspond to configurations evaluated on the same 32 input packages and diffusion-noise tensors. The time axis uses the median per-input duration; the error axis uses static BF16. Clock-state and execution-path differences limit a purely precision-based causal interpretation.

B. AGGREGATE AGREEMENT MISSES AN INPUT-LEVEL REGRESSION

Table II and Fig. 1 show a latency-fidelity tradeoff within A. Dynamic and static mixed configurations reached 81.76 and 67.70 ms, reductions of 25.58% and 38.38% from static BF16. Their minimum input cosines were 0.928032 and 0.965292. Five dynamic-mix inputs and two static-mix inputs fell below 0.99; both configurations also failed the aggregate cosine requirement.

The vision-only result illustrates a different limitation. Its aggregate cosine of 0.999560 and MAE of 0.005076 both met the aggregate thresholds. One input nevertheless had cosine 0.987160, below the input floor. The discrepancy was visible only after preserving input-level metrics. The associated 105.63 ms median was only 3.86% below the BF16 median, indicating a small recorded timing benefit alongside a localized numerical regression.

Dynamic FFN and decoder NVFP4 retained minimum input cosines above 0.99 on this sequence. Their recorded medians were 102.39 and 101.30 ms, while normalized MAEs were 0.008535 and 0.015921. These are sequence-specific numerical tradeoffs; task success was not measured.

An expanded component screen contains 281 configuration records and 257 distinct normalized/native output pairs for the same A input hash. Eight pairs pass the aggregate criteria but fail the minimum-input floor. This screen is exploratory configuration selection on exposed inputs, not an independent test population.

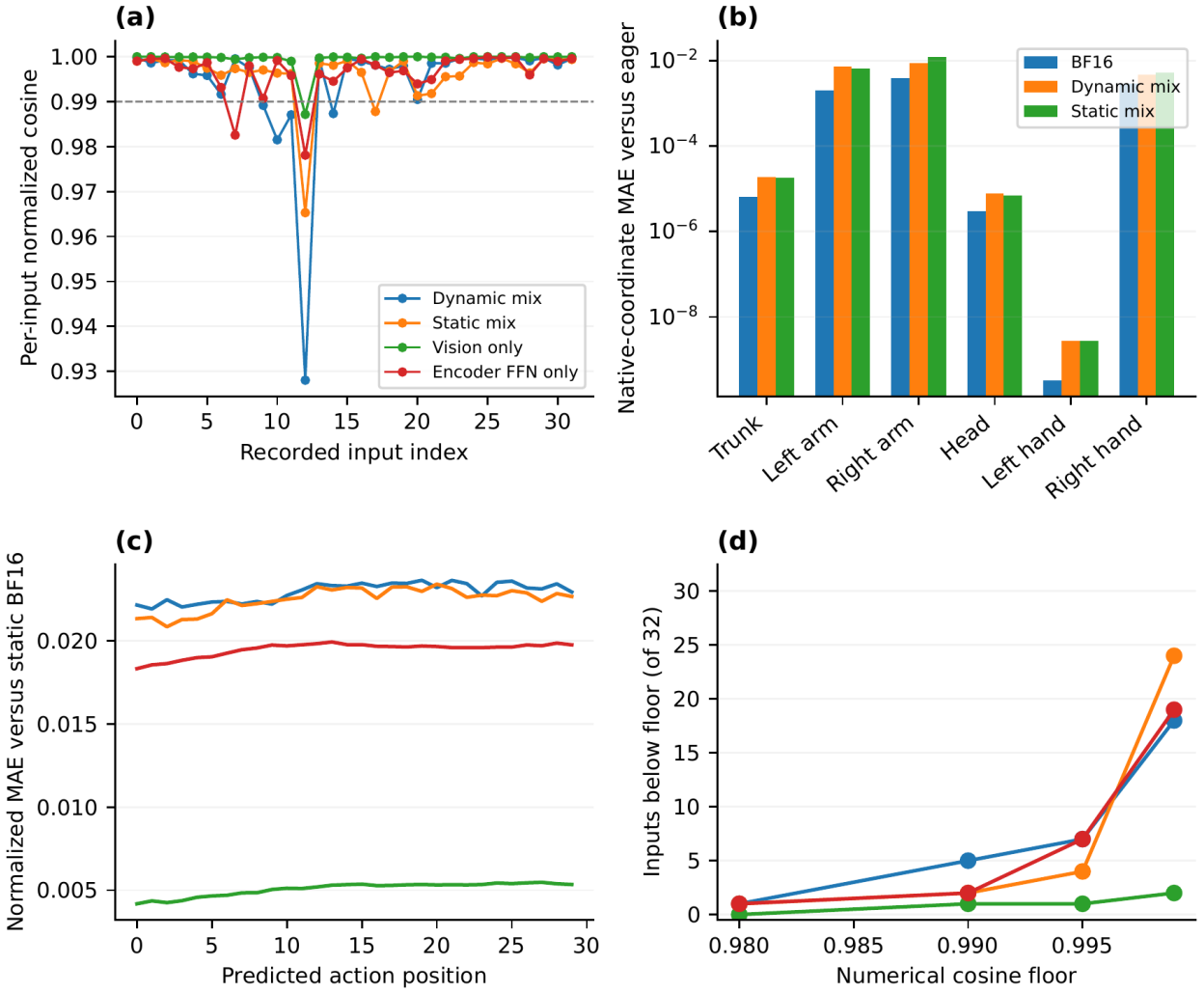


FIGURE 2. Input, coordinate-group, and predicted-position discrepancies for group A. Dashed lines denote numerical screening floors. Native-coordinate group errors use the eager reference and need not share physical units. Predicted positions belong to an action chunk; they are not independently executed future states. Threshold counts describe numerical agreement on the same 32 inputs.

C. DISCREPANCY IS HETEROGENEOUS ACROSS INPUTS AND COORDINATES

Figure 2(a) localizes the strongest mixed-precision disagreement near input 12. Across native-coordinate groups, the arms and right hand had larger discrepancies than the trunk, head, and nearly constant left-hand outputs. This heterogeneity makes a single pooled native-coordinate error insufficient for interpreting the control implications.

The horizon profiles in Fig. 2(c) showed different discrepancy levels across configurations. Errors within a predicted chunk are not evidence of temporal error accumulation during physical execution. The observed pattern can reflect the policy's computation and its action normalization, without any closed-loop rollout having occurred.

Deleting one input left dynamic-mix MAE between 0.020858 and 0.023383, compared with its full-sequence value of 0.022945. Static-mix MAE remained between 0.021668 and 0.022873. Thus, the aggregate discrepancy persisted when the most influential individual input was removed. Eight-input block deletion produced wider ranges of 0.017984–0.026102 and 0.019930–0.024622, respectively. The wider block ranges expose dependence on sequence composition.

D. OUTPUT REPEATABILITY AND TIMING VARIATION

Three B dynamic-FFN executions produced identical normalized and native actions, with timing medians of 100.43-100.66 ms. Other executions with the same output tensors reached 114.99 ms. Reproducible numerical outputs therefore did not ensure an equally stable timing result under the available execution conditions.

Two A decoder executions also produced identical actions but had timing medians of 101.30 and 115.57 ms. This difference prevents assigning runtime cost solely from output identity. Clock state, implementation revision, and process conditions were not recorded completely enough to separate their contributions.

E. INPUT-DEPENDENT PERFORMANCE OF PRECISION PROFILES

TABLE III. Whole-policy comparisons for B and D. Each group uses its own reference and timing boundary. D pairing is based on ordered identifiers, with incomplete input/noise authentication.

Configuration	Median ms	MAE	Cosine	Min. input
B: BF16	128.37	0	1	1
B: Dynamic FP8	82.30–82.64	0.00883	0.999559	0.997630
B: Static FP8	64.97–65.48	0.00807	0.999704	0.998757
B: Static AWQ	56.59–57.53	0.01377	0.999007	0.995538
D: PyTorch	294.30	0.00000	1.000000	1.000000
D: Static BF16	132.19	0.00430	0.999927	0.999560
D: Dynamic FP8	92.81	0.00647	0.999881	0.999461

B uses 32 inputs and 30-position chunks; ranges span three execution medians with identical outputs. D uses 115 inputs and 60-position chunks. Recording and scene dependence preclude a pooled independent-trial count.

B compares dynamic FP8, static FP8, and static AWQ with its BF16 implementation (Table III). Each family produced identical actions across three executions. All passed aggregate and minimum-input criteria; the lowest minimum input cosine was 0.995538 for AWQ. Thus, A does not justify a general rejection of mixed precision. Differences in checkpoint revision, calibration, and observations prevent isolating the cause of the A-B contrast.

D also showed favorable normalized agreement (Fig. 3). Dynamic FP8 had MAE 0.006468 and minimum input cosine 0.999461 against PyTorch; static BF16 had MAE 0.004301 and minimum cosine 0.999560. Against static BF16, dynamic FP8 had MAE 0.004194 and minimum cosine 0.999516. The 45/60/10 strata remain separate in the supplementary tables because identifier matching does not resolve complete input/noise equality.

The D reference timing vector has median 294.30 ms, whereas a companion report assigns its PyTorch reference 154.77 ms. We use the timing vector and its corresponding output arrays. Static-BF16 and dynamic-FP8 medians are 132.19 and 92.81 ms. We exclude the mismatched report baseline rather than combine its speed ratio with another reference.

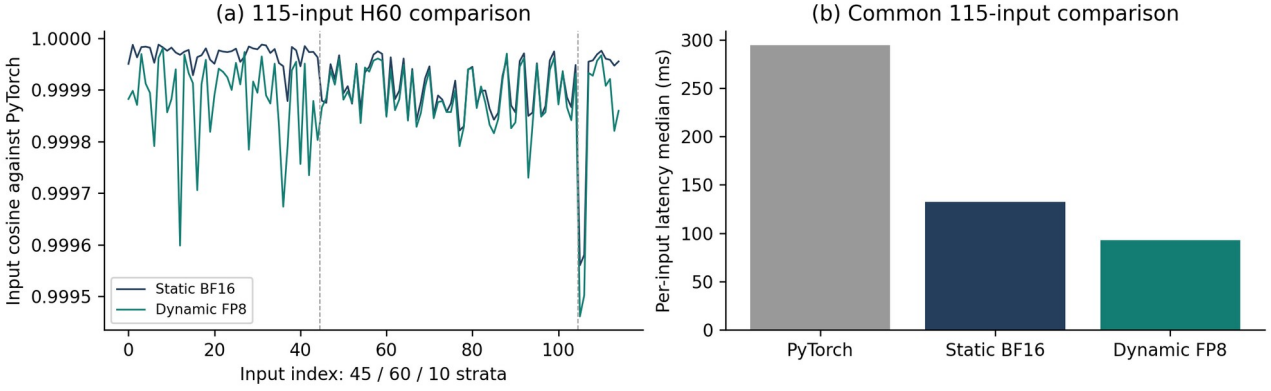


FIGURE 3. Group D with 115 inputs and 60-position action chunks. The 45/60/10 strata comprise overlapping validation identifiers, additional recording inputs, and diagnostic scenes. Input cosine uses PyTorch as reference; bars show the corresponding median per-input durations. This identifier-paired group is separate from A/B, which have complete input hashes, and from the J60 summaries.

F. SIXTY-POSITION FORWARD-CALL BENCHMARK

TABLE IV. J60 benchmark summaries. Fidelity covers 45 validation inputs. Timing covers 50 repetitions of the first prepared input following ten warmups, using CUDA events.

Profile	Median (ms)	P95 (ms)	P99 (ms)	Cosine	MAE	Max. error	Min. input cosine
Static BF16	119.95	132.76	133.15	1.000000	0.000000	0.000000	1.000000
Dynamic FP8	80.71	81.71	81.92	0.999905	0.004852	0.150391	0.998878
Dynamic FP8 + decoder NVFP4	77.12	78.05	78.32	0.999805	0.009396	0.167969	0.999025
Static FP8 + encoder AWQ NVFP4	53.31	54.10	54.35	0.999276	0.012234	0.347656	0.990386

Normalized errors use static BF16 as reference. Raw output and timing vectors for J60 are unavailable; these values are summary-level experiment results. D is a separate experiment and does not fill this missing-data boundary.

J60 uses the specified 96/16/45 calibration, selection, and validation inputs. Validation fixture checksums agree with the experiment summaries. Table IV compares four precision profiles. Because only summary measurements are available for this group, its fidelity and timing values cannot support new input-wise or inferential calculations.

Static FP8 with encoder AWQ NVFP4 reached aggregate cosine 0.999276, MAE 0.012234, and minimum input cosine 0.990386. Median forward-call latency was 53.31 ms, compared with 119.95 ms for static BF16. Three technical repeats preserved the reported fidelity and produced speed ratios of 2.2379-2.2513. The repeats reuse the validation inputs; they do not constitute 135 robot trials.

The J60 numerical and timing populations differ. Fidelity covers 45 validation inputs, while timing repeats the first prepared input 50 times (Fig. 4). Static AWQ is the fastest tested profile satisfying the specified numerical screen, but its minimum cosine exceeds 0.99 by only 0.000386. A stricter floor can therefore change the eligible profile.

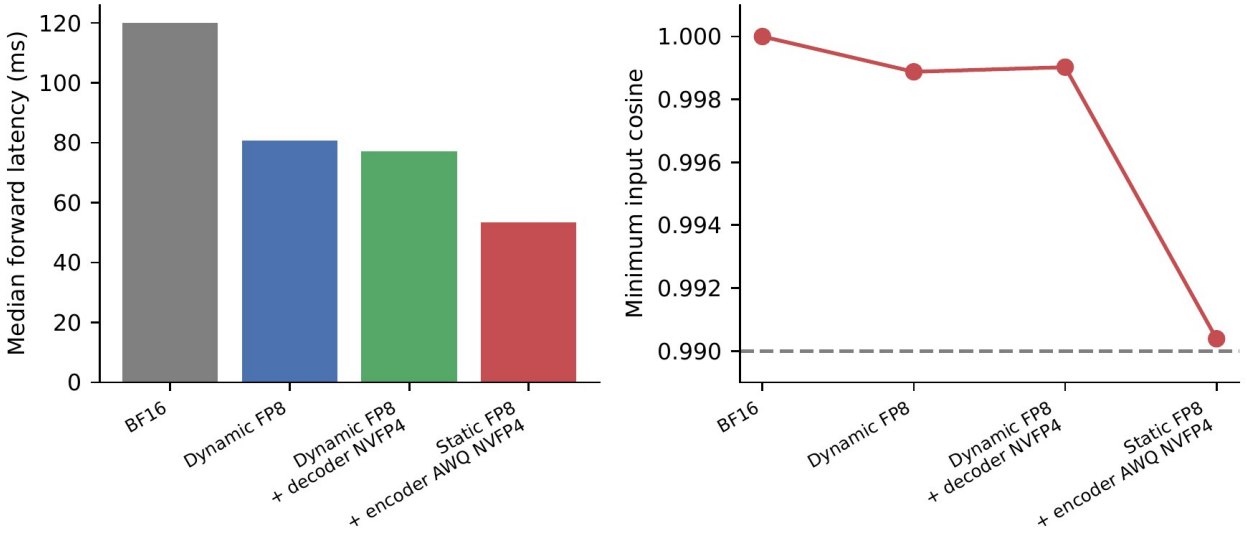


FIGURE 4. J60 forward-call timing and numerical agreement. The timing panel summarizes 50 CUDA-event repetitions of one prepared input; the fidelity panel covers 45 validation inputs. Model loading, sensor acquisition, preparation, and robot execution lie outside the forward timer. The dashed cosine floor is a numerical screening criterion.

G. NATIVE-OUTPUT COSINE CAN CONCEAL MILLIMETRE-SCALE FLANGE DIFFERENCES

TABLE V. Native-action and nominal flange discrepancies over 32 inputs and 50 predicted positions. E uses the BF16-labeled H20 reference; F uses its JAX reference. F has incomplete input/noise linkage and a gripper-column conflict.

Cohort	Configuration	Pooled cosine	Min. input cosine	Left max. (mm)	Right max. (mm)	Left max. (degrees)	Right max. (degrees)	Inputs within gate
E	PyTorch BF16 label	0.999999760	0.999999302	3.068	0.004	0.510	<0.001	29/32
E	PyTorch FP32 label	0.999999605	0.999998909	4.750	0.006	0.571	<0.001	27/32
E	TensorRT split	0.999999645	0.999999168	4.867	0.006	0.596	<0.001	21/32
E	Graph + vision BF16	0.999999639	0.999999156	3.541	0.005	0.549	<0.001	27/32
F	PyTorch BF16 label	0.999889956	0.998246877	6.487	9.041	0.613	1.440	26/32
F	TensorRT FP16 label	0.999953065	0.999124130	7.170	8.575	0.523	1.486	23/32
F	TensorRT FP8 label	0.999837586	0.998233708	19.926	15.314	2.074	2.538	3/32

The screen requires both flanges to remain within 3 mm and 0.5° at every position of an input. Counts quantify reference agreement, not successful episodes. Precision labels specify intended paths and do not certify every operator dtype. Geometry provides nominal target discrepancy rather than measured robot tracking.

E local PyTorch had pooled native cosine 0.999999760 against the H20 reference, with maximum left-flange differences of 3.068 mm and 0.510° (Table V). TensorRT split had cosine 0.999999645, with 4.867 mm and 0.596° maxima. Right-arm position maxima were only 0.004 and 0.006 mm. The task activates the left arm, so nearly unchanged coordinates contribute strongly to pooled agreement.

Twenty-nine PyTorch inputs and 21 split inputs met the 3-mm/0.5° screen at all predicted positions. Against local PyTorch instead of H20, split produced 5.314 mm and 0.650° maxima, with 24 eligible inputs. The FP32-labeled candidate reached a 4.750-mm maximum against H20. Higher nominal precision did not uniformly improve agreement with that reference.

F TensorRT FP8 had pooled cosine 0.999838 but nominal left/right position maxima of 19.926/15.314 mm and orientation maxima of 2.074°/2.538°. Only three inputs met the joint screen, compared with 26 for BF16 and 23 for FP16. However, zero-based fixture column 7 conflicts with a preprocessing rule that assumes gripper width in [0,0.09] m. Incomplete input/noise linkage and run revision prevent attributing these differences specifically to quantization. F is excluded from configuration recommendations.

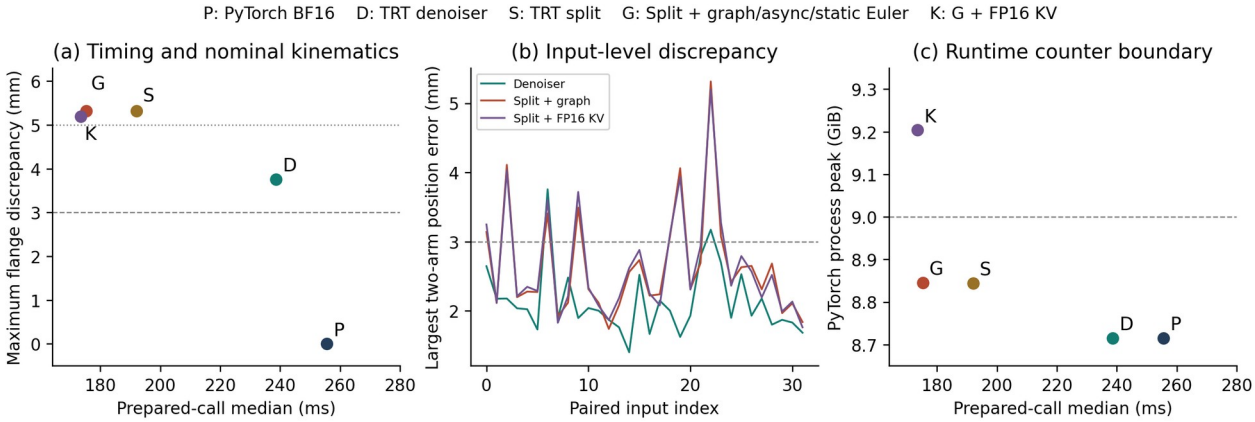


FIGURE 5. Joint configuration evaluation for the E optimization experiment. All points use the local PyTorch reference and the same 32 observations with matched noise. Panels relate prepared-call median to maximum two-arm flange discrepancy, display input-level position errors, and compare PyTorch process peaks. The counter is incomplete for TensorRT allocations. Dashed budgets are illustrative numerical constraints, not physical safety limits.

H. JOINT SELECTION UNDER DISCREPANCY AND RESOURCE BUDGETS

Under the first E timer, split reached 209.31 ms against 287.35 ms for PyTorch (27.16% reduction), while the counter increased from 8.16 to 8.84 GiB. Adding vision BF16 to the graph ladder reduced 513.01 to 290.36 ms, which still failed to improve on ordinary PyTorch.

The optimized E comparison uses one local reference, matched observations and noise, ten warmups, and 200 measured calls per configuration (Table VI; Fig. 5). TensorRT denoiser reduced median prepared-call latency from 255.51 to 238.64 ms (6.60%), with maximum two-arm flange difference 3.756 mm and 30/32 inputs within 3 mm/0.5°. Split plus graph reached 175.27 ms (31.40%), with 5.314 mm and 0.650° maxima and 24/32 eligible inputs. Its minimum input cosine remained 0.999998942. The PyTorch process peak increased from 8.715 to 8.845 GiB.

Table VI compares five representative configurations under a 9-GiB counter ceiling, with agreement required over all 32 inputs. The selected configurations are PyTorch at 3 mm/0.5°, denoiser at 5 mm/0.5°, and split plus graph at 6 mm/0.75°. FP16-cache split is 0.99% faster than graph-enabled split but exceeds the ceiling at 9.204 GiB. Expanding the screen to all 11 record identifiers preserves these backend families; the fastest denoiser alias reaches 238.03 ms (6.84% reduction; Supplement S5). Without raw timing vectors, uncertainty in alias rankings cannot be estimated. P99 screening preserved the three representative choices (Supplement S5).

TABLE VI. Joint E optimization comparison with matched observations and noise. Each candidate uses 32 inputs, H50/D26, ten warmups, and 200 measured calls.

Configuration	P50 ms	Reduction %	Native MAE	Min. cosine	Left max. mm	Left max. deg.	3 mm / 0.5°	Peak GiB
PyTorch BF16	255.51	0.00	0.000000	1.000000000	0.000	0.000	32/32	8.715
TRT denoiser	238.64	6.60	0.000291	0.999999760	3.756	0.414	30/32	8.715
TRT split	192.02	24.85	0.000414	0.999998942	5.314	0.650	24/32	8.844
TRT split + graph	175.27	31.40	0.000414	0.999998942	5.314	0.650	24/32	8.845
TRT split + FP16 KV	173.54	32.08	0.000414	0.999998871	5.196	0.671	24/32	9.204

Local PyTorch is the common reference. Graph-enabled split also requests the asynchronous engine and static Euler integration; FP16 KV additionally changes cache precision. The PyTorch process peak can include loading and omits some TensorRT allocations. Timing includes preprocessing, prediction, postprocessing and CPU output transfer.

I. SERIALIZED WEIGHT STORAGE AND RUNTIME COUNTERS

The BF16 packed file occupies 6,706,757,645 bytes. Keeping its encoder AWQ NVFP4 sidecar adds 963,271,463 bytes, producing 7,670,029,108 bytes in total, or 14.36% more storage (Fig. 6). The source checkpoint occupies 8,287,136,146 bytes. A serialization change and retention of an additional sidecar are different storage comparisons.

The build manifest specifies an SM110/CUDA13/Python 3.12 native-operations interface and a sidecar checksum. Its revision and calibration count differ from the J60 timing build. Consequently, storage is an independently specified packaging measurement rather than a jointly verified point on the J60 latency frontier. Neither serialized bytes nor a PyTorch counter establishes total device memory or energy reduction.

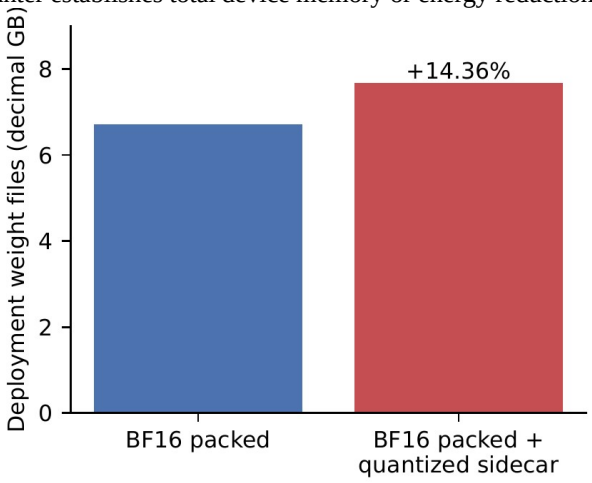


FIGURE 6. Serialized weight storage for the specified packed build. Both bars exclude shared metadata, normalization statistics, and tokenizer files. Retention of the BF16 packed weights and quantized sidecar increases distribution size. These bytes are not runtime memory measurements and are not paired with another build's timing result.

J. CACHE COMPRESSION AND COMPLETE-CYCLE COSTS HAVE DIFFERENT SCOPES

FP8 key-value tensors halve logical cache bytes before page padding and workspace overhead. Median attention latency decreased by 12.5% and 10.9% for the two 24-query cases but increased by 14.0% for the 240-query case (Fig. 7). Including quantization also increased append cost relative to prequantized values. Attention cosines of 0.999243-0.999264 characterize these synthetic kernel cases, not complete-policy action agreement.

The FP8-core/BF16-cache stream had median steady-cycle duration 2.838 s (P95 3.013 s), versus median inference duration 2.023 s. Mean cache-update duration accounted for 27.99% of mean cycle duration. Every steady cycle exceeded the configured 0.8-s action horizon in this benchmark schedule. This result does not test asynchronous control or physical deadline failures.

PyTorch allocated and reserved peaks were 21.40 and 22.81 GiB. GPU-rail integration yielded median 96.36 J per steady cycle, range 91.92-101.92 J, and P95 99.58 J. Time-weighted rail power was 33.91 W. Left-held integration changed individual estimates by at most 0.73%, without resolving sensor uncertainty. No matched whole-policy BF16 stream was measured for comparison, so these values do not establish memory or energy savings.

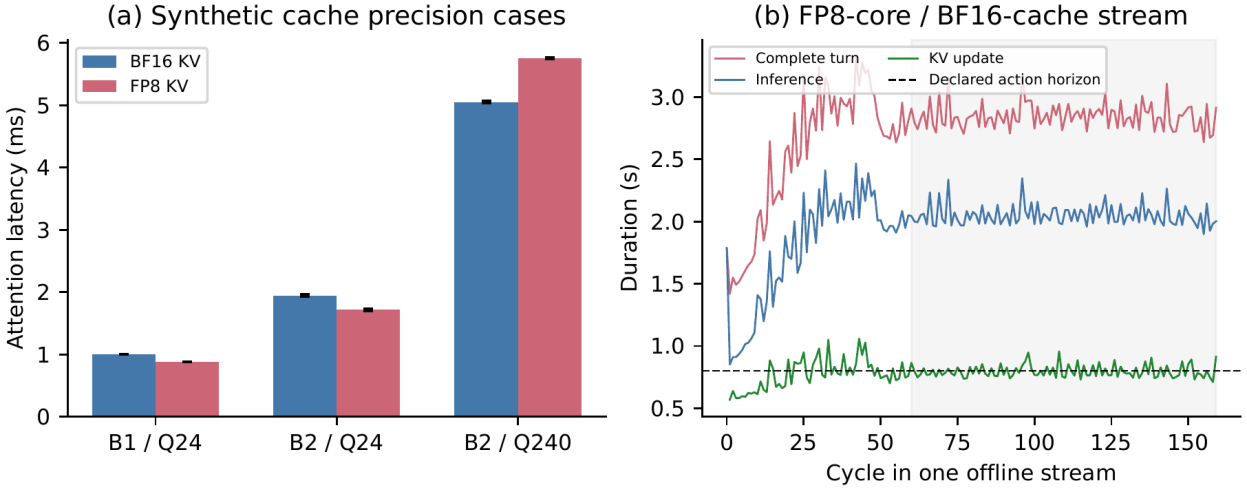


FIGURE 7. Cache precision and complete-cycle experiments with separate workloads. Kernel bars compare BF16 and FP8 key-value tensors with BF16 queries; whiskers span five repeat means. The stream uses an FP8 core and BF16 caches, with cycles 60-159 highlighted. Cache update is additional to inference. Kernel cases were not substituted into the stream, so the panels do not imply a whole-policy precision speedup.

V. DISCUSSION

A. CONFIGURATION CHOICE DEPENDS ON THE ACCEPTANCE BOUNDARY

The experiments favor selective acceleration over a universal lowest-precision setting. In A, the fastest mixed configurations fail the numerical screen, while dynamic FFN offers a smaller latency reduction with stronger agreement. In optimized E, the preferred accelerated backend changes between denoiser and graph-enabled split when the permitted nominal discrepancy increases. This is a practical configuration rule conditional on the tested inputs and resource counter.

These findings complement the action-sensitive motivations of SQIL and QVLA [3], [4]. Input-level similarity identifies localized numerical changes, and nominal kinematics expresses their joint-space consequences in robot units. Neither measure establishes that a state is task-critical. Connecting numerical outliers to physical failure requires an independently measured closed-loop outcome.

Reference choice is part of the decision. BF16 outputs define a computational comparator rather than a ground-truth controller. The eager/static comparison and E cross-device/local comparisons change the discrepancy assigned to a candidate. The D report mismatch further illustrates the need to bind a latency claim to the exact reference output. Backend, graph, preprocessing, and clock effects remain plausible alternative explanations for differences not isolated by the experiment.

B. ARITHMETIC PRECISION AND SYSTEM RESOURCE COST

Different resource constraints select different operating points. Sidecar retention increases distribution size; graph-enabled split increases the measured allocator peak; a smaller cache can slow one attention shape. Resource benefits therefore require the same workload and an explicit accounting category. A deployment budget expressed as total device memory cannot be enforced using the PyTorch counter alone.

HoloQ-VLA and CHASE-VLA use different precision scopes and timers [5], [6]. Our results should inform measurement and configuration selection rather than a numerical ranking against those methods. The contribution is the joint finite-input screen and its explicit boundaries, including cases where faster execution fails the specified agreement requirement.

C. REPETITION SHOULD PRESERVE ITS EXPERIMENTAL UNIT

The distinction between fidelity coverage and repeated timing preserves the experimental unit emphasized by benchmarking guidance [14], [15]. Observation samples and timing repetitions characterize different sources of variation; neither supplies a count of successful robot episodes.

An eligible inference configuration should identify its reference, observation package, diffusion noise, action schema, timer, and resource category. Input-level fidelity and nominal flange discrepancy provide complementary constraints. The strongest latency reduction is useful only within the budget and input coverage under which the configuration remains eligible.

D. LIMITATIONS AND FOLLOW-UP EXPERIMENTS

A/B contain short, dependent cup-transfer sequences; E contains one bottle-transfer context with incomplete per-input episode membership. D overlaps some J60 validation labels and lacks complete noise authentication. These groups do not constitute independent tasks or a previously unexposed generalization set. The illustrative kinematic budgets have no validated mapping to a manipulation tolerance.

J60 raw output and timing vectors are unavailable, limiting its analysis to summary quantities. Group A timings lack uniformly specified synchronization. The optimized timings in E preserve summary percentiles but not raw repetitions, and its memory counter can include loading. Complete run-to-source revisions and synchronized power or clock records are missing for several groups. These limitations restrict causal attribution and uncertainty estimates without invalidating the computed output discrepancies.

No closed-loop task-success, physical tracking, or safety outcome is reported. URDF-based flange differences describe predicted targets under nominal geometry. Calibration, contacts, compliance, and hand-object interactions remain unmeasured. F additionally has an unresolved input-schema conflict and is excluded from configuration selection. The separate stream has no matched precision baseline for comparative resource or energy claims.

A discriminating next experiment would use a frozen checkpoint and common timer on new task recordings, then measure whether the numerical and kinematic budgets predict physical performance. Matched baseline and candidate telemetry would resolve total-memory and energy comparisons. These measurements would test whether the finite-input selection rule transfers to control outcomes.

VI. CONCLUSION

We experimentally evaluated mixed-precision robot policy inference through paired outputs, explicit timing boundaries, and nominal joint geometry. Aggregate similarity alone was insufficient for configuration selection. The four-camera comparison showed that denoiser acceleration produced smaller nominal kinematic discrepancies, whereas graph-enabled split acceleration provided a larger latency reduction. The resulting configuration screen chooses the fastest of five representative candidates satisfying declared discrepancy and counter budgets over all evaluated inputs. Its recommendation changes with those budgets and remains specific to the checkpoint and observations. Serialized storage, allocator peaks, cache bytes, and rail measurements retain separate meanings. Closed-loop task equivalence and comparative energy savings remain unestablished.

DATA AND CODE AVAILABILITY

Derived metric tables, figure inputs, and a standalone verification script are available at https://github.com/jensenzhou47/robot-policy-inference-quantization-data/releases/tag/v1.0.1. Additional joint-selection tables and reproducible analysis routines accompany this submission. Checksums link the derived tables to their source outputs. Raw robot observations, model checkpoints, and complete native-action tensors are retained by the author and are outside the public derived-data release. Requests for underlying data should be directed to the corresponding author.

REFERENCES

[1] M. J. Kim, K. Pertsch, S. Karamcheti *et al.*, "OpenVLA: An Open-Source Vision-Language-Action Model," *arXiv preprint arXiv:2406.09246*, 2024. [Online]. Available: https://arxiv.org/abs/2406.09246

[2] Physical Intelligence, K. Black *et al.*, "$\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization," *arXiv preprint arXiv:2504.16054*, 2025. [Online]. Available: https://arxiv.org/abs/2504.16054

[3] S. Park, H. Kim, S. Kim, W. Jeon, J. Yang, B. Jeon, Y. Oh, and J. Choi, "Saliency-Aware Quantized Imitation Learning for Efficient Robotic Control," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 13 140–13 150. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2025/html/Park_Saliency-Aware_Quantized_Imitation_Learning_for_Efficient_Robotic_Control_ICCV_2025_paper.html

[4] Y. Xu, Y. Yang, Z. Fan, Y. Liu, Y. Li, B. Li, and Z. Zhang, "QVLA: Not All Channels Are Equal in Vision-Language-Action Model's Quantization," *arXiv preprint arXiv:2602.03782*, 2026. [Online]. Available: https://arxiv.org/abs/2602.03782

[5] X. Wang, M. Li, S. Lyu *et al.*, "HoloQ-VLA: Uniform W4A4 Quantization of Vision-Language-Action Models," *arXiv preprint arXiv:2605.28803v3*, 2026. [Online]. Available: https://arxiv.org/abs/2605.28803v3

[6] J. Hyun, J. G. Min, G. Jung, and Y. Lee, "CHASE-VLA: Post-Training Quantization Framework for Vision-Language-Action Models with Chunk-Aware Scale Estimation," *arXiv preprint arXiv:2610.02666*, 2026. [Online]. Available: https://arxiv.org/abs/2610.02666

[7] L. Li, Q. Zhang, Y. Luo *et al.*, "Causal World Modeling for Robot Control," *arXiv preprint arXiv:2601.21998v2*, 2026. [Online]. Available: https://arxiv.org/abs/2601.21998v2

[8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," *arXiv preprint arXiv:2210.17323*, 2022. [Online]. Available: https://arxiv.org/abs/2210.17323

[9] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration," in *Proceedings of Machine Learning and Systems*, 2024. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html

[10] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 38 087–38 099. [Online]. Available: https://proceedings.mlr.press/v202/xiao23c.html

[11] Triton Project, "Block scaled matrix multiplication," 2026, accessed: Oct. 19, 2026. [Online]. Available: https://triton-lang.org/main/getting-started/tutorials/10-block-scaled-matmul.html

[12] NVIDIA, "Working with quantized types," 2026, TensorRT documentation. Accessed: Oct. 19, 2026. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/work-with-quantized-types.html

[13] Z. Ye, L. Chen, R. Lai *et al.*, "FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving," *arXiv preprint arXiv:2501.01005v2*, 2025. [Online]. Available: https://arxiv.org/abs/2501.01005v2

[14] V. J. Reddi, C. Cheng, D. Kanter *et al.*, "MLPerf Inference Benchmark," *arXiv preprint arXiv:1911.02549*, 2019. [Online]. Available: https://arxiv.org/abs/1911.02549

[15] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proceedings of the International Symposium on Memory Management*, 2013, pp. 63–74. [Online]. Available: https://kar.kent.ac.uk/33611/

[16] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep Reinforcement Learning at the Edge of the Statistical Precipice," *arXiv preprint arXiv:2108.13264*, 2021. [Online]. Available: https://arxiv.org/abs/2108.13264

[17] NVIDIA, "Tegrastats utility," 2025, Jetson Linux r38.2 documentation. Accessed: Oct. 19, 2026. [Online]. Available: https://docs.nvidia.com/jetson/archives/r38.2/DeveloperGuide/AT/JetsonLinuxDevelopmentTools/TegrastatsUtility.html

AUTHOR BIOGRAPHY

XIONGYULE ZHOU is with the School of Mechanical, Electronic and Control Engineering, Beijing Jiaotong University, Beijing, China. His research interests include robotics, embodied intelligence, and robot learning, with a focus on efficient policy inference and experimental evaluation. His work examines numerical precision, inference latency, resource use, and the nominal kinematic discrepancies of predicted robot actions. He is interested in reproducible evaluation methods for learned policies on robotic platforms.